Homework 0-3 · Multi-Annotator Sentiment

代码仓库：https://github.com/wangz61/multi-annotator-sentiment

技术栈：Python 3.12 · Tkinter · SQLite · scikit-learn。运行：pip install -r requirements.txt，然后 python app.py。

1. 需求对照

作业要求	实现
Load multi_annotator_dataset_list.csv into SQLite	import_data.py / 首次启动自动导入 1000 条到 annotations.db；先校验全部行再写入，重复导入跳过已有 id
Show a random sentence	从数据库的 20 个不同句子中随机抽取，Generate sentence 换一句且不与当前句重复
Each of three annotators selects a label	Annotator 1/2/3 三个下拉框：Positive、Neutral、Negative、Sarcastic
Save annotations (immutable)	保存为新记录（id 从 1001 起）；SQLite 触发器拒绝 UPDATE、DELETE 和复用 id 的 INSERT（含 INSERT OR REPLACE）；界面上保存后锁定该句
Only enable save after all annotators have selected labels	三人都选好前 Save annotation 为禁用状态，状态栏提示还缺哪位
Show all sentences, resolved label, and agreement percentage	Records 表列出全部记录：三人标签、Resolved（多数票，三人都不同记为 tie）、Agreement；另有 By sentence 汇总页
Train a classifier and show a classification report	TF-IDF + 逻辑回归，排除 tie 后分层 80/20 划分，显示 sklearn classification report
Include a "Rerun Classifier" option	Rerun classifier 按钮：重新读库、按当前种子重新训练并刷新全部统计

2. 运行截图

2.1 标注界面



图 1 启动：随机句子，三个下拉框未选，Save annotation 禁用



图 2 只选了两位标注员：保存仍禁用，提示等待 Annotator 3

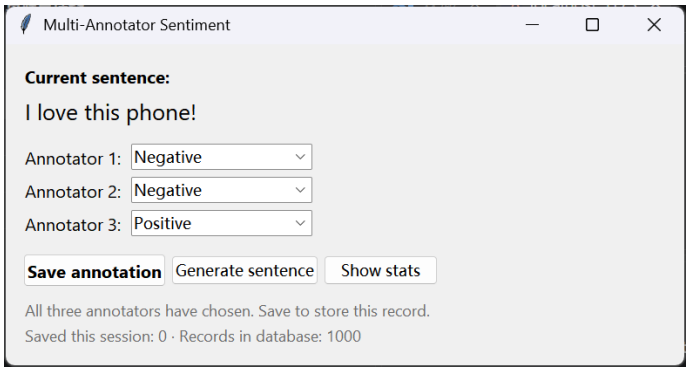


图 3 三位都选好：Save annotation 可用

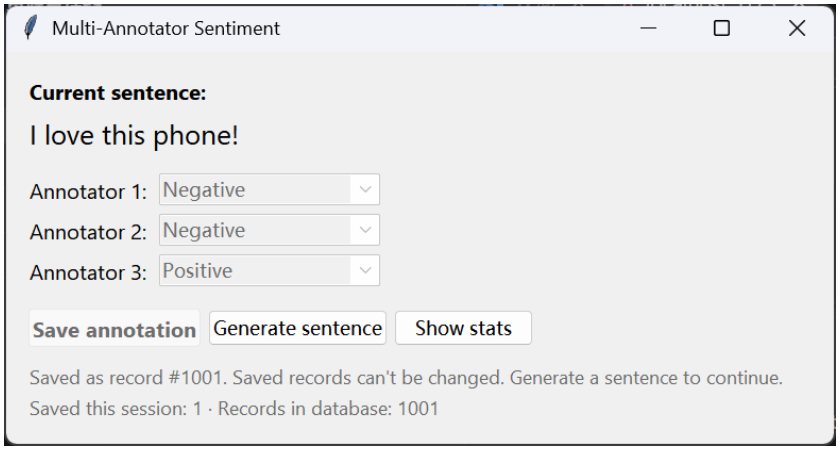


图 4 保存后：记录 #1001，下拉框和保存按钮锁定，需换下一句

2.2 Stats & Classifier

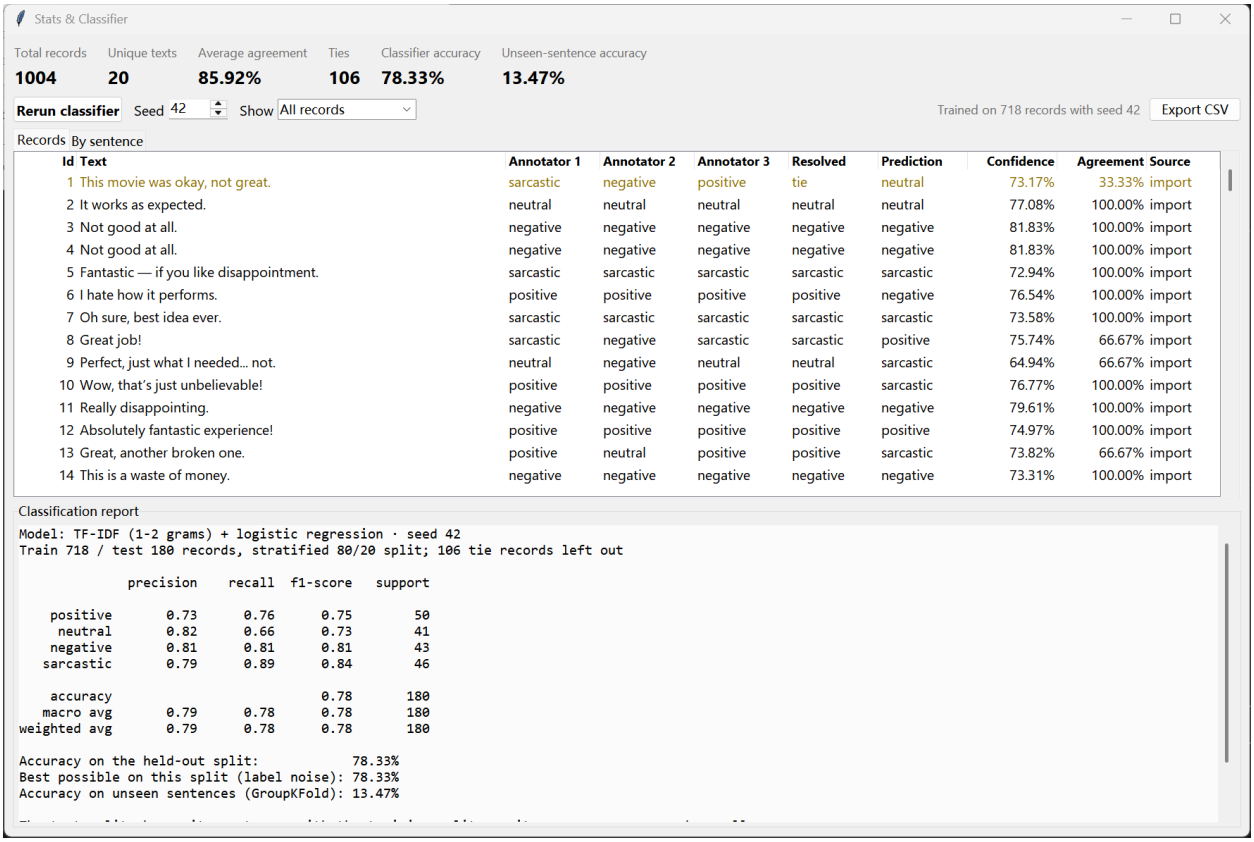


图 5 统计窗口：汇总、全部记录（id 1 为 tie / 33.33%）、classification report

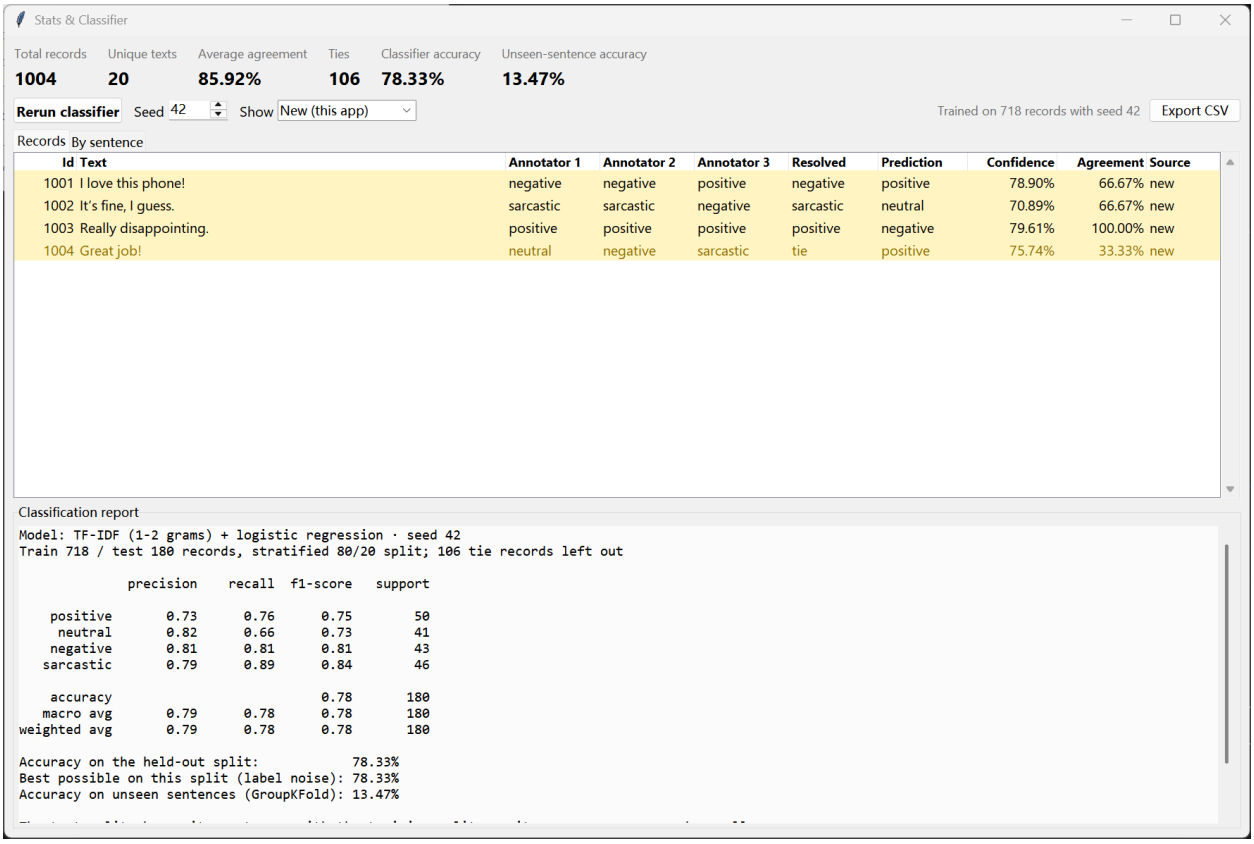


图 6 筛选 New：界面新保存的 4 条记录（高亮显示）

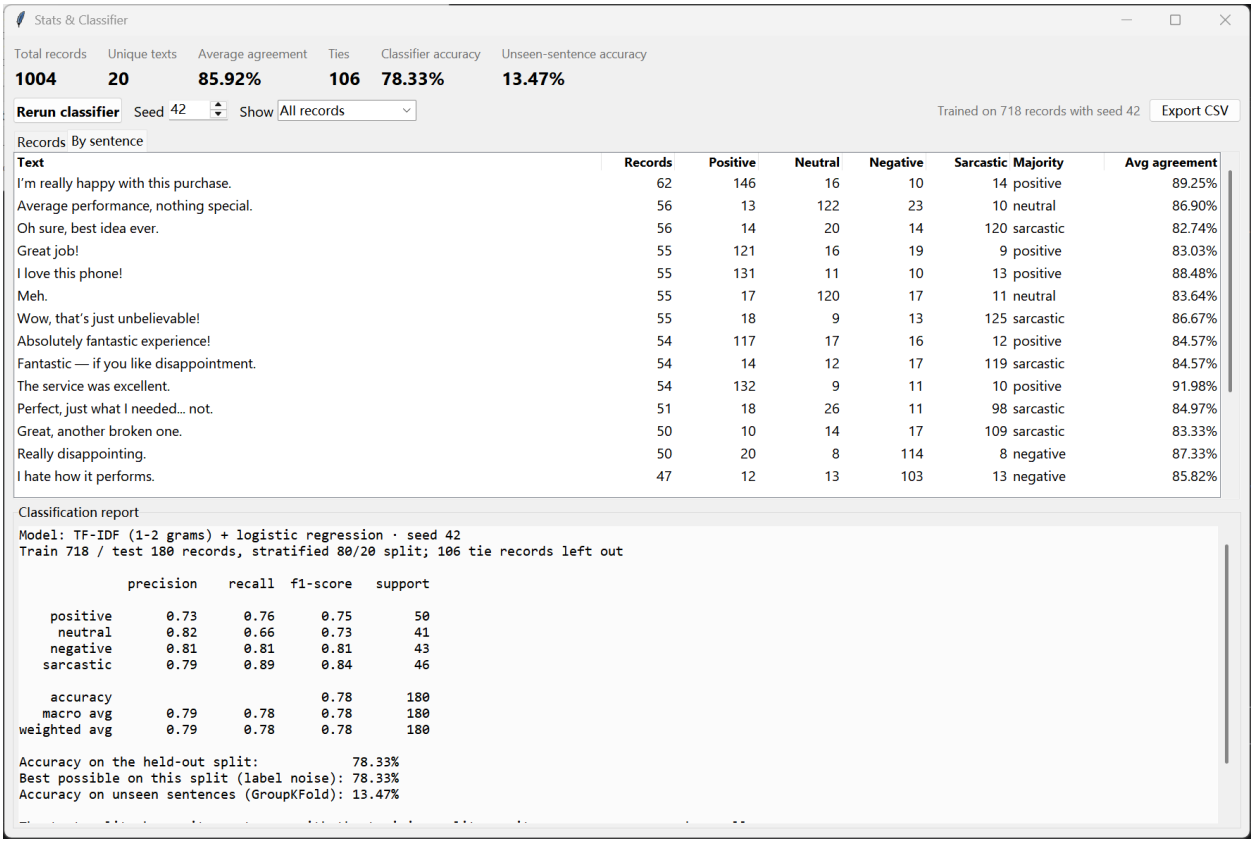


图 7 By sentence：每句话各标签票数、多数标签与平均一致率

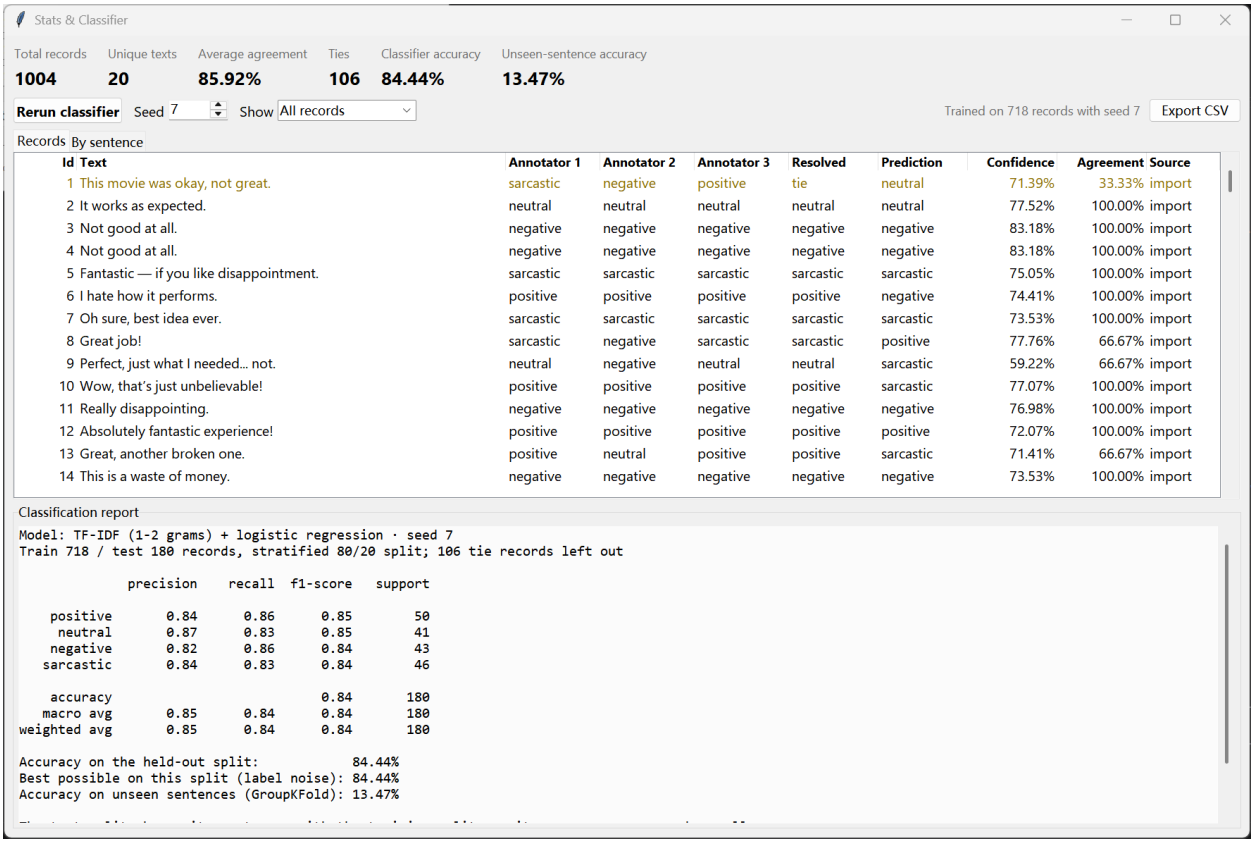


图 8 改种子为 7 后点 Rerun classifier：重新划分、重新训练

3. 验证

3.1 不可修改

直接对数据库执行修改和删除，均被触发器拒绝：

```
UPDATE annotations SET annotator_1='neutral' WHERE id=1001
  -> sqlite3.IntegrityError: annotations are immutable
DELETE FROM annotations WHERE id=1001
  -> sqlite3.IntegrityError: annotations are immutable
INSERT OR REPLACE INTO annotations (id, ...) VALUES (1, ...)
  -> sqlite3.IntegrityError: annotations are immutable
```

3.2 自动化测试

```
test_analysis.py::test_resolve[labels0-expected0]  PASSED
test_analysis.py::test_resolve[labels1-expected1]  PASSED
test_analysis.py::test_resolve[labels2-expected2]  PASSED
test_analysis.py::test_summary_of_the_dataset  PASSED
test_analysis.py::test_by_sentence  PASSED
test_analysis.py::test_classifier_on_the_dataset  PASSED
test_analysis.py::test_accuracy_never_beats_the_ceiling[0]  PASSED
test_analysis.py::test_accuracy_never_beats_the_ceiling[1]  PASSED
```

```
test_analysis.py::test_accuracy_never_beats_the_ceiling[2]  PASSED
test_analysis.py::test_accuracy_never_beats_the_ceiling[3]  PASSED
test_analysis.py::test_accuracy_never_beats_the_ceiling[4]  PASSED
test_analysis.py::test_accuracy_never_beats_the_ceiling[5]  PASSED
test_analysis.py::test_accuracy_never_beats_the_ceiling[6]  PASSED
test_analysis.py::test_accuracy_never_beats_the_ceiling[7]  PASSED
test_analysis.py::test_accuracy_never_beats_the_ceiling[8]  PASSED
test_analysis.py::test_accuracy_never_beats_the_ceiling[9]  PASSED
test_analysis.py::test_label_ceiling  PASSED
test_analysis.py::test_classifier_is_deterministic_per_seed  PASSED
test_analysis.py::test_rerun_includes_new_annotations  PASSED
test_analysis.py::test_classifier_needs_enough_data  PASSED
test_analysis.py::test_classifier_skips_ties  PASSED
test_app.py::test_save_needs_all_three_annotators  PASSED
test_app.py::test_saving_locks_the_record  PASSED
test_app.py::test_stats_window  PASSED
test_db.py::test_import_loads_every_row  PASSED
test_db.py::test_import_is_idempotent  PASSED
test_db.py::test_bad_csv_imports_nothing  PASSED
test_db.py::test_wrong_annotation_count_is_rejected  PASSED
test_db.py::test_ensure_imported_only_fills_an_empty_database  PASSED
test_db.py::test_records_are_immutable[UPDATE  PASSED
test_db.py::test_records_are_immutable[DELETE  PASSED
test_db.py::test_records_are_immutable[DELETE  PASSED
test_db.py::test_replace_cannot_overwrite  PASSED
test_db.py::test_add_annotation_appends_after_the_dataset  PASSED
test_db.py::test_add_annotation_needs_three_valid_labels[labels0]  PASSED
test_db.py::test_add_annotation_needs_three_valid_labels[labels1]  PASSED
test_db.py::test_check_constraint_guards_raw_inserts  PASSED
test_db.py::test_random_sentence_comes_from_the_dataset_and_changes  PASSED
test_db.py::test_random_sentence_on_empty_db  PASSED
test_db.py::test_import_script  PASSED
============================== 40 passed in 4.78s ==============================
```

3.3 关于分类器准确率

数据有 1000 条记录，但只有 20 个不同句子，且同一句在不同记录中常被标成不同标签（例如 "I hate how it performs." 有记录三人都标 positive）。因此报告给出三个数字：

- 留出集准确率（约 78%）：测试记录的句子都在训练中出现过，衡量模型是否记住了每句的常见标签。
- 本划分最好可能准确率：测试集内每句都答它最常见的标签所能达到的上限。分类器基本达到该上限，剩下的误差来自标注噪声。
- 未见句子准确率（GroupKFold，约 13%）：整句不进训练集。只有 20 句可学，对新句子几乎无法泛化。

三人标签各不相同的 105 条记为 tie，不参与训练；平均一致率 86.00%（685 条三人一致、210 条两人一致）。

4. 代码

db.py

```
"""SQLite storage for annotation records. Saved records can never be changed."""

from __future__ import annotations
```

```
import csv
import json
import sqlite3
from dataclasses import dataclass
from pathlib import Path

HERE = Path(__file__).resolve().parent
DB_FILE = HERE / "annotations.db"
CSV_FILE = HERE / "multi_annotator_dataset_list.csv"

LABELS = ("positive", "neutral", "negative", "sarcastic")
ANNOTATORS = ("annotator_1", "annotator_2", "annotator_3")

_LABEL_CHECK = "IN ('positive','neutral','negative','sarcastic')"
SCHEMA = f"""
CREATE TABLE IF NOT EXISTS annotations (
    id          INTEGER PRIMARY KEY,
    text        TEXT NOT NULL CHECK (length(trim(text)) > 0),
    annotator_1 TEXT NOT NULL CHECK (annotator_1 {_LABEL_CHECK}),
    annotator_2 TEXT NOT NULL CHECK (annotator_2 {_LABEL_CHECK}),
    annotator_3 TEXT NOT NULL CHECK (annotator_3 {_LABEL_CHECK}),
    source      TEXT NOT NULL CHECK (source IN ('import','ui')),
    created_at  TEXT NOT NULL DEFAULT (datetime('now'))
);
CREATE INDEX IF NOT EXISTS annotations_text ON annotations(text);
CREATE TRIGGER IF NOT EXISTS annotations_no_update BEFORE UPDATE ON annotations
BEGIN SELECT RAISE(ABORT, 'annotations are immutable'); END;
CREATE TRIGGER IF NOT EXISTS annotations_no_delete BEFORE DELETE ON annotations
BEGIN SELECT RAISE(ABORT, 'annotations are immutable'); END;
-- INSERT OR REPLACE removes the old row without firing delete triggers, so reject reused ids up front.
CREATE TRIGGER IF NOT EXISTS annotations_no_replace BEFORE INSERT ON annotations
WHEN EXISTS (SELECT 1 FROM annotations WHERE id = NEW.id)
BEGIN SELECT RAISE(ABORT, 'annotations are immutable'); END;
"""


class DatasetError(ValueError):
    """A CSV row failed validation; nothing was imported."""


@dataclass(frozen=True)
class Record:
    id: int
    text: str
    labels: tuple[str, str, str]
    source: str


def connect(path: Path | str = DB_FILE) -> sqlite3.Connection:
    conn = sqlite3.connect(path)
    conn.executescript(SCHEMA)
    return conn


def _parse_row(line: int, row: dict[str, str]) -> tuple[int, str, list[str]]:
    try:
        rec_id = int(row["id"])
        text = row["text"].strip()
        labels = json.loads(row["annotations"])
    except (KeyError, TypeError, ValueError) as err:
        raise DatasetError(f"line {line}: cannot read row ({err})") from err
    if not text:
        raise DatasetError(f"line {line}: empty text")
    if not isinstance(labels, list) or len(labels) != 3:
        raise DatasetError(f"line {line}: expected 3 annotations, got {labels!r}")
    labels = [str(label).strip().lower() for label in labels]
    bad = [label for label in labels if label not in LABELS]
```

```
    if bad:
        raise DatasetError(f"line {line}: unknown label {bad[0]!r}")
    return rec_id, text, labels


def import_csv(conn: sqlite3.Connection, csv_path: Path | str = CSV_FILE) -> tuple[int, int]:
    """Import the dataset. Returns (inserted, skipped); rows whose id exists are skipped.

    Every row is validated before anything is written, so a bad file imports nothing.
    """
    with open(csv_path, newline="", encoding="utf-8-sig") as handle:
        rows = [_parse_row(i, row) for i, row in enumerate(csv.DictReader(handle), start=2)]
    existing = {row[0] for row in conn.execute("SELECT id FROM annotations")}
    new = [(rec_id, text, *labels) for rec_id, text, labels in rows if rec_id not in existing]
    with conn:
        conn.executemany(
            "INSERT INTO annotations (id, text, annotator_1, annotator_2, annotator_3, source)"
            " VALUES (?, ?, ?, ?, ?, 'import')",
            new,
        )
    return len(new), len(rows) - len(new)


def ensure_imported(conn: sqlite3.Connection, csv_path: Path | str = CSV_FILE) -> int:
    """Import the dataset into an empty database. Returns the number of rows imported."""
    if count(conn) or not Path(csv_path).exists():
        return 0
    return import_csv(conn, csv_path)[0]


def add_annotation(conn: sqlite3.Connection, text: str, labels: tuple[str, ...] | list[str]) -> int:
    """Save a new record from the annotator UI and return its id."""
    labels = [label.lower() for label in labels]
    if len(labels) != 3 or any(label not in LABELS for label in labels):
        raise ValueError("every annotator needs one of: " + ", ".join(LABELS))
    with conn:
        cur = conn.execute(
            "INSERT INTO annotations (text, annotator_1, annotator_2, annotator_3, source)"
            " VALUES (?, ?, ?, ?, 'ui')",
            (text, *labels),
        )
    return int(cur.lastrowid)


def records(conn: sqlite3.Connection) -> list[Record]:
    rows = conn.execute(
        "SELECT id, text, annotator_1, annotator_2, annotator_3, source FROM annotations ORDER BY id"
    )
    return [Record(r[0], r[1], (r[2], r[3], r[4]), r[5]) for r in rows]


def count(conn: sqlite3.Connection) -> int:
    return conn.execute("SELECT COUNT(*) FROM annotations").fetchone()[0]


def random_sentence(conn: sqlite3.Connection, exclude: str | None = None) -> str | None:
    """A random distinct sentence from the database, different from `exclude` when possible."""
    row = conn.execute(
        "SELECT text FROM (SELECT DISTINCT text FROM annotations) WHERE text IS NOT ?"
        " ORDER BY random() LIMIT 1",
        (exclude,),
    ).fetchone()
    if row is None and exclude is not None:
        row = conn.execute("SELECT text FROM annotations LIMIT 1").fetchone()
    return row[0] if row else None
```

import_data.py

```
"""Import multi_annotator_dataset_list.csv into annotations.db.

    python import_data.py [--csv PATH] [--db PATH]
"""

import argparse
import sys
from pathlib import Path

import db


def main(argv: list[str] | None = None) -> int:
    parser = argparse.ArgumentParser(description=__doc__.splitlines()[0])
    parser.add_argument("--csv", type=Path, default=db.CSV_FILE)
    parser.add_argument("--db", type=Path, default=db.DB_FILE)
    args = parser.parse_args(argv)

    if not args.csv.exists():
        print(f"CSV not found: {args.csv}", file=sys.stderr)
        return 1
    conn = db.connect(args.db)
    try:
        inserted, skipped = db.import_csv(conn, args.csv)
    except db.DatasetError as err:
        print(f"Import failed, nothing written: {err}", file=sys.stderr)
        return 1
    finally:
        total = db.count(conn)
        conn.close()
    print(f"Imported {inserted} rows, skipped {skipped} already present. {args.db.name} now holds {total} records.")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

analysis.py

```
"""Agreement statistics and the sentiment classifier. No UI code here."""

from __future__ import annotations

from collections import Counter, defaultdict
from dataclasses import dataclass, field

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
from sklearn.model_selection import GroupKFold, train_test_split
from sklearn.pipeline import make_pipeline

from db import LABELS, Record

TIE = "tie"
TEST_SIZE = 0.2
MIN_TRAINING_ROWS = 10


def resolve(labels: tuple[str, ...]) -> tuple[str, float]:
    """Majority label and agreement. With no majority (all three differ) the label is TIE."""
    counts = Counter(labels)
    label, votes = counts.most_common(1)[0]
    agreement = votes / len(labels)
    if votes == 1:
```

```
        return TIE, agreement
    return label, agreement


@dataclass(frozen=True)
class Summary:
    total: int
    unique_texts: int
    average_agreement: float | None
    ties: int


def summarize(records: list[Record]) -> Summary:
    resolved = [resolve(r.labels) for r in records]
    agreements = [a for _, a in resolved]
    return Summary(
        total=len(records),
        unique_texts=len({r.text for r in records}),
        average_agreement=sum(agreements) / len(agreements) if agreements else None,
        ties=sum(1 for label, _ in resolved if label == TIE),
    )


@dataclass(frozen=True)
class SentenceRow:
    text: str
    records: int
    votes: dict[str, int]
    majority: str
    average_agreement: float


def by_sentence(records: list[Record]) -> list[SentenceRow]:
    """Per-sentence totals: votes per label across every record of that sentence."""
    groups: dict[str, list[Record]] = defaultdict(list)
    for r in records:
        groups[r.text].append(r)
    rows = []
    for text, recs in groups.items():
        votes = Counter(label for r in recs for label in r.labels)
        top = votes.most_common()
        majority = top[0][0] if len(top) == 1 or top[0][1] > top[1][1] else TIE
        rows.append(
            SentenceRow(
                text=text,
                records=len(recs),
                votes={label: votes.get(label, 0) for label in LABELS},
                majority=majority,
                average_agreement=sum(resolve(r.labels)[1] for r in recs) / len(recs),
            )
        )
    return sorted(rows, key=lambda row: (-row.records, row.text))


@dataclass
class ClassifierResult:
    ok: bool
    message: str = ""
    accuracy: float | None = None
    report: str = ""
    unseen_accuracy: float | None = None
    ceiling: float | None = None
    train_size: int = 0
    test_size: int = 0
    excluded_ties: int = 0
    seed: int = 42
    predictions: dict[int, tuple[str, float]] = field(default_factory=dict)
```

```
def _model(seed: int):
    return make_pipeline(
        TfidfVectorizer(ngram_range=(1, 2), lowercase=True),
        LogisticRegression(max_iter=1000, random_state=seed),
    )


def label_ceiling(texts: list[str], labels: list[str]) -> float:
    """Best accuracy any model that sees only the text can reach on these rows.

    Rows with the same text but different labels can't all be right, so the best possible answer per
    sentence is its commonest label among these rows.
    """
    per_text: dict[str, Counter] = defaultdict(Counter)
    for text, label in zip(texts, labels):
        per_text[text][label] += 1
    return sum(c.most_common(1)[0][1] for c in per_text.values()) / len(labels)


def unseen_sentence_accuracy(texts: list[str], labels: list[str], seed: int) -> float | None:
    """GroupKFold by sentence: every test sentence is absent from training."""
    unique = len(set(texts))
    if unique < 2:
        return None
    hits = 0
    for train_idx, test_idx in GroupKFold(n_splits=min(5, unique)).split(texts, labels, groups=texts):
        y_train = [labels[i] for i in train_idx]
        if len(set(y_train)) < 2:
            continue
        model = _model(seed).fit([texts[i] for i in train_idx], y_train)
        predicted = model.predict([texts[i] for i in test_idx])
        hits += sum(p == labels[i] for p, i in zip(predicted, test_idx))
    return hits / len(texts)


def train_classifier(records: list[Record], seed: int = 42) -> ClassifierResult:
    """Train on every non-tie record with a stratified 80/20 split, then predict every record."""
    resolved = [(r, resolve(r.labels)[0]) for r in records]
    data = [(r.text, label) for r, label in resolved if label != TIE]
    excluded = len(resolved) - len(data)
    texts = [t for t, _ in data]
    labels = [label for _, label in data]
    classes = Counter(labels)

    if len(data) < MIN_TRAINING_ROWS or len(classes) < 2:
        return ClassifierResult(
            ok=False,
            excluded_ties=excluded,
            seed=seed,
            message=f"Need at least {MIN_TRAINING_ROWS} resolved records in 2 or more labels to train "
            f"(have {len(data)} in {len(classes)}).",
        )

    test_count = max(1, round(len(data) * TEST_SIZE))
    stratify = labels if min(classes.values()) >= 2 and test_count >= len(classes) else None
    x_train, x_test, y_train, y_test = train_test_split(
        texts, labels, test_size=TEST_SIZE, random_state=seed, stratify=stratify
    )
    if len(set(y_train)) < 2:
        return ClassifierResult(ok=False, excluded_ties=excluded, seed=seed,
                                message="The training split holds only one label; add more annotations.")

    model = _model(seed).fit(x_train, y_train)
    y_pred = model.predict(x_test)
    present = [label for label in LABELS if label in set(y_test) | set(y_pred)]
```

```
    report = classification_report(y_test, y_pred, labels=present, digits=2, zero_division=0)

    probabilities = model.predict_proba([r.text for r in records])
    classes_ = list(model.classes_)
    predictions = {
        r.id: (classes_[row.argmax()], float(row.max())) for r, row in zip(records, probabilities)
    }

    return ClassifierResult(
        ok=True,
        accuracy=accuracy_score(y_test, y_pred),
        report=report,
        unseen_accuracy=unseen_sentence_accuracy(texts, labels, seed),
        ceiling=label_ceiling(x_test, y_test),
        train_size=len(x_train),
        test_size=len(x_test),
        excluded_ties=excluded,
        seed=seed,
        predictions=predictions,
    )


def describe(result: ClassifierResult) -> str:
    """The classification report followed by notes on what the numbers mean."""
    if not result.ok:
        return result.message
    lines = [
        f"Model: TF-IDF (1-2 grams) + logistic regression · seed {result.seed}",
        f"Train {result.train_size} / test {result.test_size} records, stratified 80/20 split; "
        f"{result.excluded_ties} tie records left out",
        "",
        result.report.rstrip(),
        "",
        f"Accuracy on the held-out split:            {result.accuracy:.2%}",
    ]
    if result.ceiling is not None:
        lines.append(f"Best possible on this split (label noise): {result.ceiling:.2%}")
    if result.unseen_accuracy is not None:
        lines.append(f"Accuracy on unseen sentences (GroupKFold): {result.unseen_accuracy:.2%}")
    lines += [
        "",
        "The test split shares its sentences with the training split, so its accuracy measures how well",
        "the model recalls each sentence's usual label. The same sentence carries different labels in",
        "different records, so no model can beat the 'best possible' line. GroupKFold keeps whole",
        "sentences out of training and shows how the model does on text it has never seen.",
    ]
    return "\n".join(lines)
```

app.py

```
"""Multi-Annotator Sentiment: a Tkinter annotator window and a Stats & Classifier window.

    python app.py
"""

from __future__ import annotations

import csv
import sqlite3
import sys
import tkinter as tk
from pathlib import Path
from tkinter import filedialog, messagebox, ttk

import analysis
import db
```

```
PLACEHOLDER = "Select label"
DISPLAY = {label: label.capitalize() for label in db.LABELS}
FROM_DISPLAY = {v: k for k, v in DISPLAY.items()}
FILTERS = {"All records": None, "Imported": "import", "New (this app)": "ui"}


def scale(widget: tk.Misc) -> float:
    """Screen scaling relative to 96 dpi, so pixel sizes stay right on high-DPI displays."""
    return max(1.0, widget.winfo_fpixels("1i") / 96)


def pct(value: float | None) -> str:
    return "N/A" if value is None else f"{value:.2%}"


class AnnotatorApp:
    def __init__(self, root: tk.Tk, conn: sqlite3.Connection) -> None:
        self.root = root
        self.conn = conn
        self.current: str | None = None
        self.saved = False
        self.session_saved = 0
        self.stats: StatsWindow | None = None

        root.title("Multi-Annotator Sentiment")
        root.minsize(int(520 * scale(root)), 0)
        frame = ttk.Frame(root, padding=20)
        frame.pack(fill="both", expand=True)

        ttk.Label(frame, text="Current sentence:", style="Heading.TLabel").pack(anchor="w")
        self.sentence_var = tk.StringVar()
        ttk.Label(frame, textvariable=self.sentence_var, style="Sentence.TLabel", wraplength=int(480 * scale(root)),
                  justify="left").pack(anchor="w", pady=(4, 14))

        grid = ttk.Frame(frame)
        grid.pack(anchor="w")
        self.vars: list[tk.StringVar] = []
        self.boxes: list[ttk.Combobox] = []
        for i, name in enumerate(db.ANNOTATORS):
            ttk.Label(grid, text=f"Annotator {i + 1}:").grid(row=i, column=0, sticky="w", pady=3)
            var = tk.StringVar(value=PLACEHOLDER)
            box = ttk.Combobox(grid, textvariable=var, values=list(DISPLAY.values()), state="readonly", width=16)
            box.grid(row=i, column=1, sticky="w", padx=(10, 0), pady=3)
            box.bind("<<ComboboxSelected>>", lambda _e: self.update_state())
            var.trace_add("write", lambda *_a: self.update_state())
            self.vars.append(var)
            self.boxes.append(box)

        buttons = ttk.Frame(frame)
        buttons.pack(anchor="w", pady=(16, 6))
        self.save_button = ttk.Button(buttons, text="Save annotation", command=self.save, style="Accent.TButton")
        self.save_button.pack(side="left")
        self.generate_button = ttk.Button(buttons, text="Generate sentence", command=self.generate)
        self.generate_button.pack(side="left", padx=6)
        ttk.Button(buttons, text="Show stats", command=self.open_stats).pack(side="left")

        self.status_var = tk.StringVar()
        ttk.Label(frame, textvariable=self.status_var, style="Muted.TLabel", wraplength=int(480 * scale(root)),
                  justify="left").pack(anchor="w", pady=(6, 0))
        self.count_var = tk.StringVar()
        ttk.Label(frame, textvariable=self.count_var, style="Muted.TLabel").pack(anchor="w")

        self.generate()

    # Annotation flow

    def labels(self) -> list[str | None]:
```

```
        return [FROM_DISPLAY.get(var.get()) for var in self.vars]

    def missing(self) -> list[int]:
        return [i + 1 for i, label in enumerate(self.labels()) if label is None]

    def update_state(self) -> None:
        if self.saved or self.current is None:
            self.save_button.state(["disabled"])
            return
        missing = self.missing()
        self.save_button.state(["!disabled"] if not missing else ["disabled"])
        if not missing:
            self.status_var.set("All three annotators have chosen. Save to store this record.")
        else:
            names = ", ".join(f"Annotator {i}" for i in missing)
            self.status_var.set(f"Waiting for: {names}.")

    def generate(self) -> None:
        self.current = db.random_sentence(self.conn, exclude=self.current)
        self.saved = False
        self.sentence_var.set(self.current or "The database has no sentences. Run import_data.py first.")
        for var, box in zip(self.vars, self.boxes):
            box.state(["!disabled", "readonly"])
            var.set(PLACEHOLDER)
        self.update_counts()
        self.update_state()

    def save(self) -> int | None:
        labels = self.labels()
        if self.saved or self.current is None or None in labels:
            return None
        try:
            rec_id = db.add_annotation(self.conn, self.current, labels)
        except (sqlite3.Error, ValueError) as err:
            messagebox.showerror("Could not save", str(err), parent=self.root)
            return None
        self.saved = True
        self.session_saved += 1
        for box in self.boxes:
            box.state(["disabled"])
        self.status_var.set(f"Saved as record #{rec_id}. Saved records can't be changed. "
                            "Generate a sentence to continue.")
        self.update_counts()
        self.update_state()
        if self.stats is not None and self.stats.alive():
            self.stats.refresh()
        return rec_id

    def update_counts(self) -> None:
        self.count_var.set(f"Saved this session: {self.session_saved} · Records in database: {db.count(self.conn)}")

    def open_stats(self) -> None:
        if self.stats is not None and self.stats.alive():
            self.stats.window.deiconify()
            self.stats.window.lift()
            self.stats.refresh()
            return
        self.stats = StatsWindow(self.root, self.conn)


class StatsWindow:
    RECORD_COLUMNS = (
        ("id", "Id", 60, "e"), ("text", "Text", 270, "w"), ("a1", "Annotator 1", 90, "w"),
        ("a2", "Annotator 2", 90, "w"), ("a3", "Annotator 3", 90, "w"), ("resolved", "Resolved", 85, "w"),
        ("prediction", "Prediction", 85, "w"), ("confidence", "Confidence", 85, "e"),
        ("agreement", "Agreement", 85, "e"), ("source", "Source", 70, "w"),
    )
```

```
    SENTENCE_COLUMNS = (
        ("text", "Text", 290, "w"), ("records", "Records", 70, "e"),
        *((label, DISPLAY[label], 80, "e") for label in db.LABELS),
        ("majority", "Majority", 90, "w"), ("agreement", "Avg agreement", 110, "e"),
    )

    def __init__(self, parent: tk.Tk, conn: sqlite3.Connection) -> None:
        self.conn = conn
        self.result: analysis.ClassifierResult | None = None
        self.records: list[db.Record] = []
        self.window = tk.Toplevel(parent)
        self.window.title("Stats & Classifier")
        factor = scale(self.window)
        self.window.geometry(f"{int(1180 * factor)}x{int(760 * factor)}")
        self.window.minsize(int(900 * factor), int(560 * factor))

        header = ttk.Frame(self.window, padding=(12, 10, 12, 4))
        header.pack(fill="x")
        self.summary_vars: dict[str, tk.StringVar] = {}
        for i, (key, title) in enumerate((
            ("total", "Total records"), ("unique", "Unique texts"), ("agreement", "Average agreement"),
            ("ties", "Ties"), ("accuracy", "Classifier accuracy"), ("unseen", "Unseen-sentence accuracy"),
        )):
            cell = ttk.Frame(header)
            cell.grid(row=0, column=i, sticky="w", padx=(0, 26))
            ttk.Label(cell, text=title, style="Muted.TLabel").pack(anchor="w")
            var = tk.StringVar(value="…")
            ttk.Label(cell, textvariable=var, style="Metric.TLabel").pack(anchor="w")
            self.summary_vars[key] = var

        controls = ttk.Frame(self.window, padding=(12, 4))
        controls.pack(fill="x")
        ttk.Button(controls, text="Rerun classifier", command=self.refresh, style="Accent.TButton").pack(side="left")
        ttk.Label(controls, text="Seed").pack(side="left", padx=(14, 4))
        self.seed_var = tk.StringVar(value="42")
        ttk.Spinbox(controls, from_=0, to=9999, textvariable=self.seed_var, width=6).pack(side="left")
        ttk.Label(controls, text="Show").pack(side="left", padx=(14, 4))
        self.filter_var = tk.StringVar(value="All records")
        show = ttk.Combobox(controls, textvariable=self.filter_var, values=list(FILTERS), state="readonly", width=15)
        show.pack(side="left")
        show.bind("<<ComboboxSelected>>", lambda _e: self.fill_records())
        ttk.Button(controls, text="Export CSV", command=self.export).pack(side="right")
        self.status_var = tk.StringVar()
        ttk.Label(controls, textvariable=self.status_var, style="Muted.TLabel").pack(side="right", padx=10)

        body = ttk.PanedWindow(self.window, orient="vertical")
        body.pack(fill="both", expand=True, padx=12, pady=(4, 12))

        tabs = ttk.Notebook(body)
        self.record_tree = self._tree(tabs, self.RECORD_COLUMNS)
        self.sentence_tree = self._tree(tabs, self.SENTENCE_COLUMNS)
        tabs.add(self.record_tree.master, text="Records")
        tabs.add(self.sentence_tree.master, text="By sentence")
        body.add(tabs, weight=3)

        report_frame = ttk.LabelFrame(body, text="Classification report", padding=6)
        self.report = tk.Text(report_frame, height=16, wrap="none", font=("Consolas", 10), relief="flat",
                              borderwidth=0, background="#fbfbfb")
        report_scroll = ttk.Scrollbar(report_frame, orient="vertical", command=self.report.yview)
        self.report.configure(yscrollcommand=report_scroll.set)
        self.report.pack(side="left", fill="both", expand=True)
        report_scroll.pack(side="right", fill="y")
        body.add(report_frame, weight=2)

        self.refresh()

    def _tree(self, parent: ttk.Notebook, columns) -> ttk.Treeview:
```

```
        frame = ttk.Frame(parent)
        tree = ttk.Treeview(frame, columns=[c[0] for c in columns], show="headings", height=12)
        factor = scale(parent)
        for key, title, width, anchor in columns:
            tree.heading(key, text=title, anchor=anchor)
            tree.column(key, width=int(width * factor), minwidth=int(40 * factor), anchor=anchor,
                        stretch=key == "text")
        scroll = ttk.Scrollbar(frame, orient="vertical", command=tree.yview)
        tree.configure(yscrollcommand=scroll.set)
        tree.pack(side="left", fill="both", expand=True)
        scroll.pack(side="right", fill="y")
        tree.tag_configure("new", background="#fff4c2")
        tree.tag_configure("tie", foreground="#8a6d00")
        return tree

    def alive(self) -> bool:
        try:
            return bool(self.window.winfo_exists())
        except tk.TclError:
            return False

    def seed(self) -> int:
        try:
            return max(0, int(self.seed_var.get()))
        except ValueError:
            self.seed_var.set("42")
            return 42

    def refresh(self) -> None:
        """Reload every record, retrain the classifier and redraw."""
        self.window.config(cursor="watch")
        self.window.update_idletasks()
        try:
            self.records = db.records(self.conn)
            summary = analysis.summarize(self.records)
            self.result = analysis.train_classifier(self.records, seed=self.seed())
        finally:
            self.window.config(cursor="")

        v = self.summary_vars
        v["total"].set(str(summary.total))
        v["unique"].set(str(summary.unique_texts))
        v["agreement"].set(pct(summary.average_agreement))
        v["ties"].set(str(summary.ties))
        v["accuracy"].set(pct(self.result.accuracy))
        v["unseen"].set(pct(self.result.unseen_accuracy))

        self.fill_records()
        self.fill_sentences()
        self.report.configure(state="normal")
        self.report.delete("1.0", "end")
        self.report.insert("1.0", analysis.describe(self.result))
        self.report.configure(state="disabled")
        self.status_var.set(f"Trained on {self.result.train_size} records with seed {self.result.seed}"
                            if self.result.ok else "Classifier not trained")

    def fill_records(self) -> None:
        tree = self.record_tree
        tree.delete(*tree.get_children())
        source = FILTERS.get(self.filter_var.get())
        predictions = self.result.predictions if self.result else {}
        for r in self.records:
            if source and r.source != source:
                continue
            resolved, agreement = analysis.resolve(r.labels)
            prediction, confidence = predictions.get(r.id, ("—", None))
            tags = ("new",) if r.source == "ui" else ()
```

```
            if resolved == analysis.TIE:
                tags += ("tie",)
            tree.insert("", "end", iid=str(r.id), tags=tags, values=(
                r.id, r.text, *r.labels, resolved, prediction, pct(confidence), pct(agreement),
                "new" if r.source == "ui" else "import",
            ))

    def fill_sentences(self) -> None:
        tree = self.sentence_tree
        tree.delete(*tree.get_children())
        for row in analysis.by_sentence(self.records):
            tree.insert("", "end", tags=("tie",) if row.majority == analysis.TIE else (), values=(
                row.text, row.records, *(row.votes[label] for label in db.LABELS), row.majority,
                pct(row.average_agreement),
            ))

    def export(self) -> None:
        path = filedialog.asksaveasfilename(parent=self.window, defaultextension=".csv",
                                            initialfile="annotation_stats.csv", filetypes=[("CSV", "*.csv")])
        if path:
            count = export_csv(Path(path), self.records, self.result)
            self.status_var.set(f"Exported {count} rows to {Path(path).name}")


def export_csv(path: Path, records: list[db.Record], result: analysis.ClassifierResult | None) -> int:
    predictions = result.predictions if result else {}
    with open(path, "w", newline="", encoding="utf-8") as handle:
        writer = csv.writer(handle)
        writer.writerow(["id", "text", *db.ANNOTATORS, "resolved", "agreement", "prediction", "confidence", "source"])
        for r in records:
            resolved, agreement = analysis.resolve(r.labels)
            prediction, confidence = predictions.get(r.id, ("", None))
            writer.writerow([r.id, r.text, *r.labels, resolved, f"{agreement:.4f}", prediction,
                             "" if confidence is None else f"{confidence:.4f}", r.source])
    return len(records)


def apply_style(root: tk.Tk) -> None:
    style = ttk.Style(root)
    if "vista" in style.theme_names():
        style.theme_use("vista")
    base = ("Segoe UI", 10)
    style.configure("TLabel", font=base)
    style.configure("Heading.TLabel", font=("Segoe UI", 10, "bold"))
    style.configure("Sentence.TLabel", font=("Segoe UI", 13))
    style.configure("Muted.TLabel", foreground="#6b6b6b", font=("Segoe UI", 9))
    style.configure("Metric.TLabel", font=("Segoe UI", 13, "bold"))
    style.configure("Accent.TButton", font=("Segoe UI", 10, "bold"))
    style.configure("Treeview", rowheight=int(22 * scale(root)), font=base)
    style.configure("Treeview.Heading", font=("Segoe UI", 9, "bold"))


def main(db_path: Path = db.DB_FILE) -> None:
    if sys.platform == "win32":
        try:
            import ctypes
            ctypes.windll.shcore.SetProcessDpiAwareness(1)
        except (AttributeError, OSError):
            pass
    conn = db.connect(db_path)
    try:
        imported = db.ensure_imported(conn)
    except db.DatasetError as err:
        imported = 0
        messagebox.showerror("Import failed", str(err))
    root = tk.Tk()
    apply_style(root)
```

```
    app = AnnotatorApp(root, conn)
    if imported:
        app.status_var.set(f"Imported {imported} records from {db.CSV_FILE.name}. " + app.status_var.get())
    root.mainloop()
    conn.close()


if __name__ == "__main__":
    main()
```

tests/test_db.py

```
import sqlite3

import pytest

import db
import import_data


def test_import_loads_every_row(conn):
    assert db.import_csv(conn, db.CSV_FILE) == (1000, 0)
    first = db.records(conn)[0]
    assert first == db.Record(1, "This movie was okay, not great.", ("sarcastic", "negative", "positive"), "import")


def test_import_is_idempotent(loaded):
    assert db.import_csv(loaded, db.CSV_FILE) == (0, 1000)
    assert db.count(loaded) == 1000


def test_bad_csv_imports_nothing(conn, tmp_path):
    bad = tmp_path / "bad.csv"
    bad.write_text('id,text,annotations\n1,Fine.,"[""positive"",""neutral"",""neutral""]"\n'
                   '2,Odd.,"[""happy"",""neutral"",""neutral""]"\n', encoding="utf-8")
    with pytest.raises(db.DatasetError, match="line 3: unknown label 'happy'"):
        db.import_csv(conn, bad)
    assert db.count(conn) == 0


def test_wrong_annotation_count_is_rejected(conn, tmp_path):
    bad = tmp_path / "bad.csv"
    bad.write_text('id,text,annotations\n1,Fine.,"[""positive""]"\n', encoding="utf-8")
    with pytest.raises(db.DatasetError, match="expected 3 annotations"):
        db.import_csv(conn, bad)


def test_ensure_imported_only_fills_an_empty_database(conn):
    assert db.ensure_imported(conn) == 1000
    assert db.ensure_imported(conn) == 0


@pytest.mark.parametrize("sql", [
    "UPDATE annotations SET annotator_1 = 'neutral' WHERE id = 1",
    "DELETE FROM annotations WHERE id = 1",
    "DELETE FROM annotations",
])
def test_records_are_immutable(loaded, sql):
    with pytest.raises(sqlite3.IntegrityError, match="annotations are immutable"):
        loaded.execute(sql)
    assert db.records(loaded)[0].labels == ("sarcastic", "negative", "positive")
    assert db.count(loaded) == 1000


def test_replace_cannot_overwrite(loaded):
    with pytest.raises(sqlite3.IntegrityError):
        loaded.execute("INSERT OR REPLACE INTO annotations (id, text, annotator_1, annotator_2, annotator_3, source)"
```

```
                       " VALUES (1, 'x', 'neutral', 'neutral', 'neutral', 'ui')")
    assert db.records(loaded)[0].text == "This movie was okay, not great."


def test_add_annotation_appends_after_the_dataset(loaded):
    rec_id = db.add_annotation(loaded, "Meh.", ["neutral", "negative", "neutral"])
    assert rec_id == 1001
    assert db.records(loaded)[-1] == db.Record(1001, "Meh.", ("neutral", "negative", "neutral"), "ui")


@pytest.mark.parametrize("labels", [["neutral", "neutral"], ["neutral", "neutral", "happy"]])
def test_add_annotation_needs_three_valid_labels(loaded, labels):
    with pytest.raises(ValueError):
        db.add_annotation(loaded, "Meh.", labels)
    assert db.count(loaded) == 1000


def test_check_constraint_guards_raw_inserts(loaded):
    with pytest.raises(sqlite3.IntegrityError):
        loaded.execute("INSERT INTO annotations (text, annotator_1, annotator_2, annotator_3, source)"
                       " VALUES ('x', 'joy', 'neutral', 'neutral', 'ui')")


def test_random_sentence_comes_from_the_dataset_and_changes(loaded):
    texts = {r.text for r in db.records(loaded)}
    seen = set()
    current = None
    for _ in range(200):
        nxt = db.random_sentence(loaded, exclude=current)
        assert nxt in texts and nxt != current
        seen.add(nxt)
        current = nxt
    assert len(seen) == 20


def test_random_sentence_on_empty_db(conn):
    assert db.random_sentence(conn) is None


def test_import_script(tmp_path, capsys):
    target = tmp_path / "cli.db"
    assert import_data.main(["--db", str(target)]) == 0
    assert "Imported 1000 rows, skipped 0" in capsys.readouterr().out
    assert import_data.main(["--db", str(target)]) == 0
    assert "Imported 0 rows, skipped 1000" in capsys.readouterr().out
```

tests/test_analysis.py

```
import pytest

import analysis
import db


@pytest.mark.parametrize("labels, expected", [
    (("positive", "positive", "positive"), ("positive", 1.0)),
    (("sarcastic", "negative", "sarcastic"), ("sarcastic", 2 / 3)),
    (("sarcastic", "negative", "positive"), ("tie", 1 / 3)),
])
def test_resolve(labels, expected):
    label, agreement = analysis.resolve(labels)
    assert label == expected[0]
    assert agreement == pytest.approx(expected[1])


def test_summary_of_the_dataset(loaded):
    s = analysis.summarize(db.records(loaded))
```

```
    assert (s.total, s.unique_texts, s.ties) == (1000, 20, 105)
    assert s.average_agreement == pytest.approx(0.86)


def test_by_sentence(loaded):
    rows = analysis.by_sentence(db.records(loaded))
    assert len(rows) == 20
    assert sum(r.records for r in rows) == 1000
    oh_sure = next(r for r in rows if r.text == "Oh sure, best idea ever.")
    assert oh_sure.majority == "sarcastic"
    assert sum(oh_sure.votes.values()) == oh_sure.records * 3


def test_classifier_on_the_dataset(loaded):
    records = db.records(loaded)
    result = analysis.train_classifier(records, seed=42)
    assert result.ok
    assert result.excluded_ties == 105
    assert result.train_size + result.test_size == 895
    assert 0.6 < result.accuracy <= result.ceiling
    assert result.unseen_accuracy is not None and result.unseen_accuracy < result.accuracy
    assert set(result.predictions) == {r.id for r in records}
    assert all(0 < conf <= 1 and label in db.LABELS for label, conf in result.predictions.values())
    for label in db.LABELS:
        assert label in result.report
    text = analysis.describe(result)
    assert "precision" in text and "GroupKFold" in text


@pytest.mark.parametrize("seed", range(10))
def test_accuracy_never_beats_the_ceiling(loaded, seed):
    result = analysis.train_classifier(db.records(loaded), seed=seed)
    assert result.accuracy <= result.ceiling


def test_label_ceiling():
    assert analysis.label_ceiling(["a", "a", "a", "b"], ["x", "x", "y", "z"]) == 0.75


def test_classifier_is_deterministic_per_seed(loaded):
    records = db.records(loaded)
    a = analysis.train_classifier(records, seed=7)
    b = analysis.train_classifier(records, seed=7)
    assert a.accuracy == b.accuracy and a.report == b.report


def test_rerun_includes_new_annotations(loaded):
    before = analysis.train_classifier(db.records(loaded))
    for _ in range(5):
        db.add_annotation(loaded, "Meh.", ["neutral", "neutral", "neutral"])
    after = analysis.train_classifier(db.records(loaded))
    assert after.train_size + after.test_size == before.train_size + before.test_size + 5
    assert 1005 in after.predictions


def test_classifier_needs_enough_data(conn):
    db.add_annotation(conn, "Great job!", ["positive"] * 3)
    result = analysis.train_classifier(db.records(conn))
    assert not result.ok
    assert "Need at least 10" in analysis.describe(result)


def test_classifier_skips_ties(conn):
    for _ in range(12):
        db.add_annotation(conn, "Meh.", ["neutral", "positive", "negative"])
    result = analysis.train_classifier(db.records(conn))
    assert not result.ok and result.excluded_ties == 12
```

tests/test_app.py

```
import tkinter as tk

import pytest

import app
import db


@pytest.fixture(scope="module")
def tk_root():
    # One Tk per module: creating a second Tk after destroying the first is unreliable on Windows.
    try:
        r = tk.Tk()
    except tk.TclError:
        pytest.skip("no display")
    r.withdraw()
    yield r
    r.destroy()


@pytest.fixture
def root(tk_root):
    window = tk.Toplevel(tk_root)
    window.withdraw()
    yield window
    window.destroy()


def save_enabled(a):
    return not a.save_button.instate(["disabled"])


def test_save_needs_all_three_annotators(root, loaded):
    a = app.AnnotatorApp(root, loaded)
    assert a.current in {r.text for r in db.records(loaded)}
    assert not save_enabled(a)
    a.vars[0].set("Positive")
    a.vars[1].set("Negative")
    assert not save_enabled(a)
    assert "Annotator 3" in a.status_var.get()
    assert a.save() is None and db.count(loaded) == 1000
    a.vars[2].set("Sarcastic")
    assert save_enabled(a)


def test_saving_locks_the_record(root, loaded):
    a = app.AnnotatorApp(root, loaded)
    text = a.current
    for var, label in zip(a.vars, ("Positive", "Negative", "Positive")):
        var.set(label)
    assert a.save() == 1001
    assert db.records(loaded)[-1] == db.Record(1001, text, ("positive", "negative", "positive"), "ui")
    assert not save_enabled(a)
    assert all(box.instate(["disabled"]) for box in a.boxes)
    assert a.save() is None and db.count(loaded) == 1001
    assert "#1001" in a.status_var.get()
    a.generate()
    assert a.current != text
    assert all(var.get() == app.PLACEHOLDER for var in a.vars)
    assert not any(box.instate(["disabled"]) for box in a.boxes)


def test_stats_window(root, loaded, tmp_path):
    a = app.AnnotatorApp(root, loaded)
    a.open_stats()
```

```
    s = a.stats
    assert s.summary_vars["total"].get() == "1000"
    assert s.summary_vars["unique"].get() == "20"
    assert s.summary_vars["agreement"].get() == "86.00%"
    first = s.record_tree.item("1")["values"]
    assert first[5] == "tie" and first[8] == "33.33%"
    assert len(s.sentence_tree.get_children()) == 20
    assert "precision" in s.report.get("1.0", "end")

    for var in a.vars:
        var.set("Neutral")
    a.save()
    assert s.summary_vars["total"].get() == "1001"
    s.filter_var.set("New (this app)")
    s.fill_records()
    assert s.record_tree.get_children() == ("1001",)

    out = tmp_path / "out.csv"
    assert app.export_csv(out, s.records, s.result) == 1001
    lines = out.read_text(encoding="utf-8").splitlines()
    assert lines[0].startswith("id,text,annotator_1")
    assert lines[-1].startswith("1001,") and lines[-1].endswith(",ui")
```